

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 1 | AULA 05 -
**INTERFACE
SEGREGATION
PRINCIPLE**

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	11
REFERÊNCIAS.....	12

EMANDA

O QUE VEM POR AÍ?

Nesta aula, vamos explorar o Princípio da Segregação de Interfaces (Interface Segregation Principle). Vamos entender a importância de criar interfaces específicas para cada cliente, evitando interfaces grandes e generalistas que forçam a implementação de métodos não necessários.



HANDS ON

Nesta aula prática, apresentaremos o Princípio da Segregação de Interfaces (ISP), o quarto dos princípios SOLID da programação orientada a objetos e design de software.



SAIBA MAIS

O Princípio da Segregação de Interfaces (ISP) afirma que os clientes não devem ser forçados a depender de interfaces que não utilizam. Em outras palavras, é melhor ter várias interfaces específicas e coesas do que uma única interface generalista e grande.

A ideia principal é evitar a criação de interfaces grandes e generalistas que forcem os clientes a implementar métodos que não são relevantes para eles. Em vez disso, devemos criar interfaces menores e mais específicas que atendam exatamente às necessidades dos clientes.

Para ficar mais claro, vamos considerar um exemplo de uma interface que representa um trabalhador e suas atividades (**Trabalhador**), para que possamos ver um fluxo que infringe esse princípio:

```
public interface Trabalhador {  
    void trabalhar();  
    void comer();  
    void dormir();  
}
```

Código 1 — Fluxo que infringe ISP
Fonte: elaborado pelo autor (2024)

Nesse exemplo, todas as classes que forem implementar essa interface serão forçados a implementar todos os seus métodos, como **trabalhar**, **comer** e **dormir**, mesmo que algumas dessas classes não precisem de todos esses métodos.

Para ficar mais claro, vejamos dois exemplos de implementação dessa interface, uma para Funcionário e outra para Robot (Robô), iniciando pela criação da classe funcionário:

```
public class Funcionario implements Trabalhador {  
    @Override  
    public void trabalhar() {  
        System.out.println("Funcionario está trabalhando");  
    }  
}
```

```
}

@Override
public void comer() {
    System.out.println("Funcionario está comendo");
}

@Override
public void dormir() {
    System.out.println("Funcionario está dormindo");
}
}
```

Código 2 — Classe Funcionário
Fonte: elaborado pelo autor (2024)

Note que, para essa classe, faz sentido implementarmos todos os métodos da nossa interface `Trabalhador`.

Agora, vamos criar a nossa classe `Robot`:

```
public class Robot implements Trabalhador {
    @Override
    public void trabalhar() {
        System.out.println("Robot está trabalhando");
    }

    @Override
    public void comer() {
        // Robots não comem, mas são forçados a implementar este método
        throw new UnsupportedOperationException("Robots não comem");
    }

    @Override
    public void dormir() {
```

```
// Robots não dormem, mas são forçados a implementar este método
throw new UnsupportedOperationException("Robots não dormem");
}
}
```

Código 3 — Classe Robot
Fonte: elaborado pelo autor (2024)

Vejamos como seria a execução dessas classes:

```
public class Main {
    public static void main(String[] args) {
        Trabalhador funcionario = new Funcionario();
        funcionario.trabalhar();
        funcionario.comer();
        funcionario.dormir();

        Trabalhador robot = new Robot();
        robot.trabalhar();

        try {
            robot.comer();
        } catch (UnsupportedOperationException e) {
            System.out.println(e.getMessage());
        }

        try {
            robot.dormir();
        } catch (UnsupportedOperationException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Código 4 — Execução das classes
Fonte: elaborado pelo autor (2024)

Resultado:

```
● (base) thiagoadriano@Thiagos-MBP aula4 % ./sages -cp /Users/thiagoadriano/Library/Apply
Funcionario está trabalhando
Funcionario está comendo
Funcionario está dormindo
Robot está trabalhando
Robots não comem
Robots não dormem
○ (base) thiagoadriano@Thiagos-MBP aula4 %
```

Figura 1 — Retorno de validação de implementação interface Trabalhador
Fonte: elaborado pelo autor (2024)

Note que a classe Robot, mesmo não precisando dos métodos comer e dormir, foi obrigada a implementá-los. Isso seria um problema, certo? Então, como podemos resolver esse problema com o ISP?

Vamos refatorar o código, dividindo a interface **Trabalhador** em interfaces mais específicas. Primeiro, criaremos uma para o método Dormir, chamada Descanso:

```
package src;

public interface Descanso {
    void dormir();
}
```

Código 5 — Descanso
Fonte: elaborado pelo autor (2024)

Agora, vamos criar outra para o método Comer, chamada Refeicao:

```
package src;

public interface Refeicao {
    void comer();
}
```

Código 6 — Refeicao
Fonte: elaborado pelo autor (2024)

Refatorando as nossas classes Funcionario e Robot, nós temos:

Funcionario:

```
package src;

public class Funcionario implements Trabalhador, Refeicao, Descanso {

    @Override
    public void trabalhar() {
        System.out.println("Funcionario está trabalhando");
    }

    @Override
    public void comer() {
        System.out.println("Funcionario está comendo");
    }

    @Override
    public void dormir() {
        System.out.println("Funcionario está dormindo");
    }
}
```

Código 7 — Refatorando Funcionario
Fonte: elaborado pelo autor (2024)

Robot:

```
package src;

public class Robot implements Trabalhador {

    @Override
    public void trabalhar() {
        System.out.println("Robot está trabalhando");
    }
}
```

Código 9 — Refatorando Robot
Fonte: elaborado pelo autor (2024)

Explicando a refatoração

Antes da refatoração, a interface **Trabalhador** estava violando o ISP ao forçar a implementação de métodos que não eram necessários para todos os trabalhadores, como no caso do **Robot**, que não precisa comer ou dormir.

Depois da refatoração:

1. Interface **Trabalhador**: mantém apenas o método **trabalhar**, garantindo que a interface seja específica para trabalhadores.
2. Interface **Refeicao**: define o método **comer**, específico para clientes que precisam dessa funcionalidade.
3. Interface **Descanso**: define o método **dormir**, específico para clientes que precisam dessa funcionalidade.
4. Classe **Funcionario**: implementa todas as interfaces necessárias (**Trabalhador**, **Comedor**, **Dorminhoco**), mantendo-se coesa e específica.
5. Classe **Robot**: implementa apenas a interface **Trabalhador**, mantendo-se específica e sem métodos desnecessários.

Benefícios da refatoração

- Modularidade: interfaces menores e específicas facilitam a compreensão e a manutenção do código.
- Redução de acoplamento: clientes não são forçados a depender de métodos que não utilizam, reduzindo o acoplamento e aumentando a flexibilidade do sistema.
- Facilidade de manutenção: mudanças em uma interface específica afetam apenas os clientes que realmente dependem dela, facilitando a evolução do sistema.
- Coesão: as interfaces e classes se mantêm focadas em uma única responsabilidade, melhorando a coesão do código.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos o Princípio da Segregação de Interfaces. Abordamos a definição do ISP, em que clientes não devem ser forçados a depender de interfaces que não utilizam. Vimos também a importância do ISP, que facilita a manutenção e a modularidade do código, criando interfaces menores e específicas.

Além disso, estudamos um exemplo prático de como refatorar uma interface que viola o ISP para uma estrutura que segue esse princípio, criando interfaces coesas e específicas. Por fim, abordamos seus benefícios, como modularidade, redução de acoplamento, facilidade de manutenção e aumento da coesão do código.

REFERÊNCIAS

CASTILHO, R. **Princípios SOLID: Princípio da Segregação de Interface (ISP)**. 2022. Disponível em: <https://robsoncastilho.com.br/2013/04/14/principios-solid-principio-da-segregacao-de-interface-isp/>. Acesso em: 07 ago. 2024.

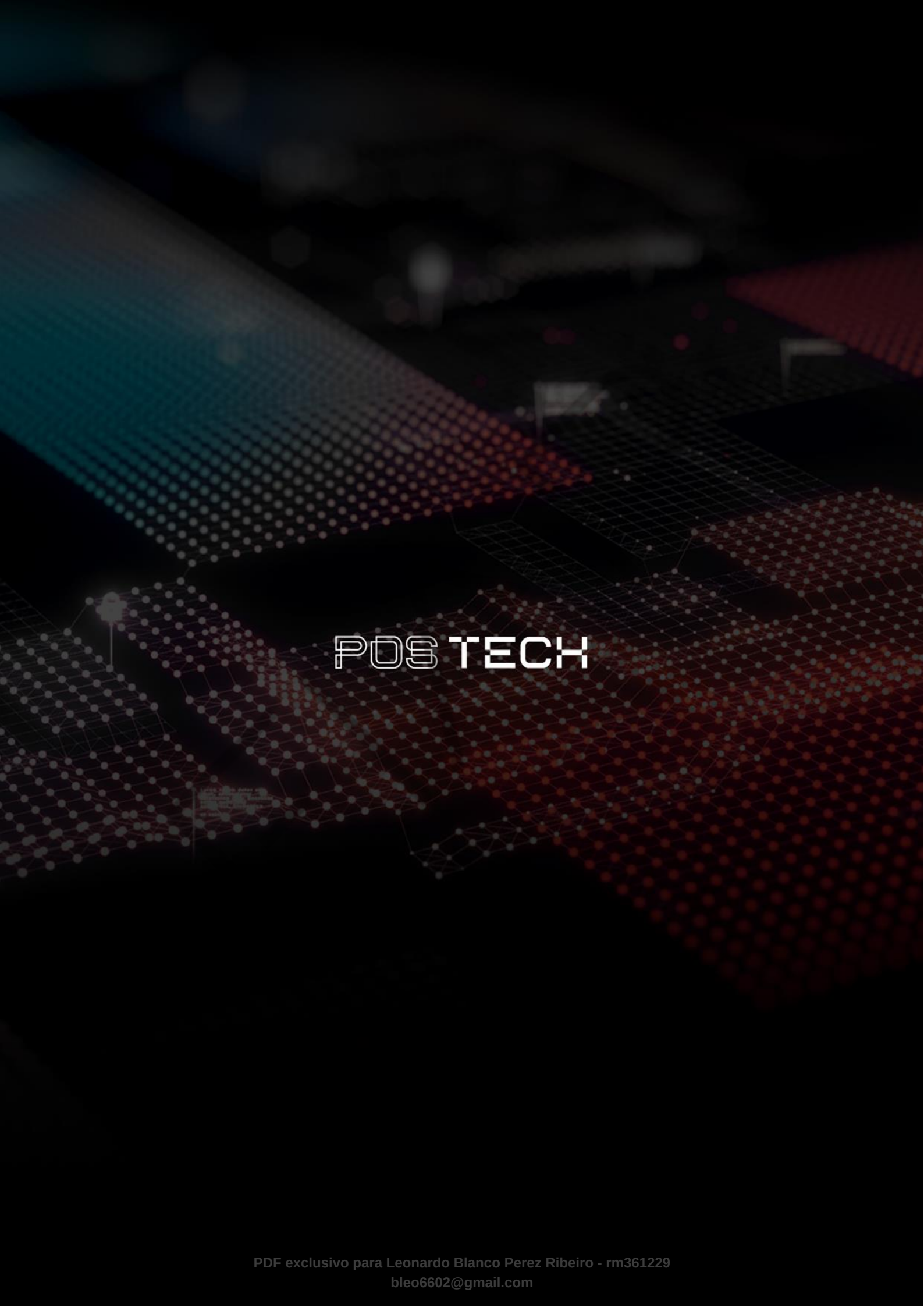
IMASTERS. **SOLID de verdade – Interface Segregation Principle (ISP)**. 2020. Disponível em: <https://imasters.com.br/arquitetura-da-informacao/solid-de-verdade-interface-segregation-principle-isp>. Acesso em: 07 ago. 2024.

MEDIUM. **Solid — I.S.P — Interface Segregation Principle**. 2019. Disponível em: <https://medium.com/@tbaragao/solid-i-s-p-interface-segregation-principle-c0b25d7dccf9>. Acesso em: 07 ago. 2024.

PALAVRAS-CHAVE

Palavras-chave: SOLID, Princípio da Segregação de Interfaces, Interface Segregation Principle, ISP.

EMSE



POSTECH